LLD Design Patterns in Java

Focused study notes: understanding, usage, interview signals, and Java code

How to use this PDF: First understand the symptom each pattern fixes. Then read the Java code. In interviews, explain the design reason before naming the pattern.

The 13 patterns covered

1. 1. Strategy
2. 2. Factory
3. 3. Observer
4. 4. State
5. 5. Singleton
6. 6. Builder
7. 7. Decorator
8. 8. Adapter
9. 9. Chain of Responsibility
10. 10. Repository
11. 11. Command
12. 12. Facade
13. 13. Composite

Pattern selection cheat sheet

Question	Answer
Multiple algorithms or business rules?	Strategy
Object creation depends on type?	Factory
One event should notify many listeners?	Observer
Behavior changes based on lifecycle status?	State
Only one shared instance should exist?	Singleton
Too many constructor parameters?	Builder
Need add-ons or feature combinations?	Decorator
Third-party interface does not match yours?	Adapter
Request passes through multiple handlers?	Chain of Responsibility
Service is polluted with DB logic?	Repository
Need undo, queue, retry, or action history?	Command
Client must call too many services?	Facade
Tree-like hierarchy?	Composite

1. Strategy Pattern

Core idea: It lets you define a family of interchangeable algorithms behind a common interface.

Question	Answer
Symptom in code	A service has many if-else branches for payment, pricing, discount, sorting, validation, or routing.
Mental model	Same task, different method. The client should depend on the interface, not the exact algorithm.
Interview sentence	I will use Strategy Pattern because it lets you define a family of interchangeable algorithms behind a common interface.

When to use it

- There are multiple ways to perform the same operation.
- You expect new algorithms to be added later.
- You want to avoid large if-else or switch statements.
- The behavior should be selected at runtime.

Where it appears in LLD

- Payment methods in BookMyShow, e-commerce, wallet apps.
- Pricing strategies in parking lot, cab booking, subscriptions.
- Splitwise split strategies: equal, exact, percentage.
- Cache eviction: LRU, LFU, FIFO.

Common mistakes

- Do not create a strategy class for tiny logic that will never vary.
- Do not let the service know details of each strategy.
- Do not combine Strategy and Factory responsibilities in one class unless the example is very small.

Java example

```
interface PaymentStrategy {
    void pay(double amount);
}

class UpiPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using UPI");
    }
}

class CardPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using Card");
    }
}

class PaymentService {
    private final PaymentStrategy strategy;

    public PaymentService(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void makePayment(double amount) {
        strategy.pay(amount);
    }
}
```

```
class Main {
    public static void main(String[] args) {
        PaymentService service = new PaymentService(new UpiPayment());
        service.makePayment(500);
    }
}
```

2. Factory Pattern

Core idea: It centralizes object creation when the exact class depends on an input type or condition.

Question	Answer
Symptom in code	Object creation logic is repeated in many places using new plus if-else or switch.
Mental model	Ask a dedicated creator for the right object. The caller should not know all concrete classes.
Interview sentence	I will use Factory Pattern because it centralizes object creation when the exact class depends on an input type or condition.

When to use it

- Object type depends on user input, enum, config, or runtime condition.
- Creation logic is likely to change.
- You want to hide concrete classes from client code.
- You want one place to add new object types.

Where it appears in LLD

- NotificationFactory for Email, SMS, Push.
- PaymentFactory for UPI, Card, Wallet.
- VehicleFactory in parking lot.
- ParserFactory for PDF, CSV, JSON, Excel.

Common mistakes

- Factory creates objects; it should not execute the full business workflow.
- Avoid string literals in serious code; prefer enums.
- Do not overuse factory when direct constructor is perfectly clear.

Java example

```
enum NotificationType {
    EMAIL, SMS, PUSH
}

interface Notification {
    void send(String message);
}

class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

class SmsNotification implements Notification {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}

class NotificationFactory {
    public static Notification create(NotificationType type) {
        switch (type) {
            case EMAIL: return new EmailNotification();
            case SMS: return new SmsNotification();
            default: throw new IllegalArgumentException("Unsupported type");
        }
```

```
    }
}

class Main {
    public static void main(String[] args) {
        Notification notification = NotificationFactory.create(NotificationType.EMAIL);
        notification.send("Order placed");
    }
}
```

3. Observer Pattern

Core idea: It lets multiple subscribers react automatically when a subject publishes an event or changes state.

Question	Answer
Symptom in code	One service directly calls email, SMS, analytics, inventory, and other services after one event.
Mental model	One publisher, many subscribers. The publisher does not need to know exact subscriber logic.
Interview sentence	I will use Observer Pattern because it lets multiple subscribers react automatically when a subject publishes an event or changes state.

When to use it

- One event should trigger multiple actions.
- Subscribers may be added or removed later.
- You want loose coupling between event producer and consumers.
- You see notification, listener, subscription, or event words in the problem.

Where it appears in LLD

- Stock price alerts.
- Order status notifications.
- YouTube channel subscribers.
- Chat app unread counts and push notifications.
- Booking confirmation email, SMS, analytics.

Common mistakes

- Do not make observers do very slow work synchronously in production.
- Do not let subject depend on concrete observer classes.
- Be careful with duplicate subscriptions.

Java example

```
import java.util.*;

interface Observer {
    void update(String message);
}

class User implements Observer {
    private final String name;

    public User(String name) {
        this.name = name;
    }

    public void update(String message) {
        System.out.println(name + " received: " + message);
    }
}

class YoutubeChannel {
    private final List<Observer> observers = new ArrayList<>();

    public void subscribe(Observer observer) {
        observers.add(observer);
    }
```

```
    public void unsubscribe(Observer observer) {
        observers.remove(observer);
    }

    public void uploadVideo(String title) {
        for (Observer observer : observers) {
            observer.update("New video: " + title);
        }
    }
}

class Main {
    public static void main(String[] args) {
        YoutubeChannel channel = new YoutubeChannel();
        channel.subscribe(new User("Rahul"));
        channel.subscribe(new User("Aman"));
        channel.uploadVideo("LLD Patterns");
    }
}
```

4. State Pattern

Core idea: It moves state-specific behavior into separate classes so an object can change behavior as its state changes.

Question	Answer
Symptom in code	A class has many conditions based on status: CREATED, PAID, SHIPPED, CANCELLED, EXPIRED.
Mental model	The object has a lifecycle. Each state controls what actions are allowed next.
Interview sentence	I will use State Pattern because it moves state-specific behavior into separate classes so an object can change behavior as its state changes.

When to use it

- Object behavior depends heavily on current status.
- Invalid transitions must be prevented.
- You have a lifecycle: order, booking, ATM, vending machine, document approval.
- The number of status-based conditions is growing.

Where it appears in LLD

- Order lifecycle in e-commerce.
- Booking lifecycle in BookMyShow.
- Vending machine states.
- ATM states.
- Inspection/report lifecycle in audit apps.

Common mistakes

- Do not use State if enum plus simple validation is enough.
- Do not spread transition rules across many services.
- Each state should not know unrelated business concerns.

Java example

```
interface OrderState {
    void next(Order order);
    String name();
}

class CreatedState implements OrderState {
    public void next(Order order) {
        order.setState(new PaidState());
    }
    public String name() { return "CREATED"; }
}

class PaidState implements OrderState {
    public void next(Order order) {
        order.setState(new ShippedState());
    }
    public String name() { return "PAID"; }
}

class ShippedState implements OrderState {
    public void next(Order order) {
        System.out.println("Order delivered");
    }
    public String name() { return "SHIPPED"; }
}
```

```
class Order {
    private OrderState state = new CreatedState();

    public void setState(OrderState state) {
        this.state = state;
    }

    public void moveNext() {
        state.next(this);
    }

    public String getStatus() {
        return state.name();
    }
}
```

5. Singleton Pattern

Core idea: It ensures exactly one instance of a class is created and reused.

Question	Answer
Symptom in code	Multiple instances of logger, config manager, or cache manager may create inconsistent behavior.
Mental model	One shared object for one shared responsibility.
Interview sentence	I will use Singleton Pattern because it ensures exactly one instance of a class is created and reused.

When to use it

- Only one instance logically makes sense.
- The object controls shared configuration or shared infrastructure.
- Creating multiple instances can cause bugs or inconsistency.
- The instance should be globally accessible or injected once.

Where it appears in LLD

- Logger.
- ConfigManager.
- CacheManager.
- DBConnectionPool.
- Application settings.

Common mistakes

- Do not make every service singleton.
- Avoid mutable global state when possible.
- In real applications, dependency injection is often cleaner than hard-coded singletons.

Java example

```
class Logger {
    private static final Logger INSTANCE = new Logger();

    private Logger() {
    }

    public static Logger getInstance() {
        return INSTANCE;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}

class Main {
    public static void main(String[] args) {
        Logger.getInstance().log("Application started");
    }
}
```

6. Builder Pattern

Core idea: It constructs complex objects step by step, especially when there are many optional fields.

Question	Answer
Symptom in code	Constructors are long, unreadable, overloaded, or full of null values.
Mental model	Readable object construction through chained methods.
Interview sentence	I will use Builder Pattern because it constructs complex objects step by step, especially when there are many optional fields.

When to use it

- Object has many fields.
- Many fields are optional.
- You want immutable objects after creation.
- Creation includes validation or step-by-step configuration.

Where it appears in LLD

- User profile object.
- HTTP request builder.
- Report builder.
- Order builder.
- Audit report builder with site details, tests, images, signature.

Common mistakes

- Do not use Builder for objects with only two or three required fields.
- Keep validation inside build() if object must be consistent.
- Avoid exposing partially built objects.

Java example

```
class User {
    private final String name;
    private final String email;
    private final String city;
    private final boolean premium;

    private User(Builder builder) {
        this.name = builder.name;
        this.email = builder.email;
        this.city = builder.city;
        this.premium = builder.premium;
    }

    static class Builder {
        private String name;
        private String email;
        private String city;
        private boolean premium;

        public Builder name(String name) {
            this.name = name;
            return this;
        }

        public Builder email(String email) {
            this.email = email;
            return this;
```

```
        }

        public Builder city(String city) {
            this.city = city;
            return this;
        }

        public Builder premium(boolean premium) {
            this.premium = premium;
            return this;
        }

        public User build() {
            return new User(this);
        }
    }
}
```

7. Decorator Pattern

Core idea: It adds behavior dynamically by wrapping an object instead of creating many subclasses.

Question	Answer
Symptom in code	You are creating many subclasses for combinations like CoffeeWithMilkAndSugarAndCream.
Mental model	Start with a base object and wrap it with add-ons.
Interview sentence	I will use Decorator Pattern because it adds behavior dynamically by wrapping an object instead of creating many subclasses.

When to use it

- Features can be combined in many ways.
- You want to add behavior without modifying the original class.
- Subclass explosion is happening.
- Add-ons should be stackable.

Where it appears in LLD

- Coffee or pizza toppings.
- Notification channels: email plus SMS plus push.
- Text formatting: bold, italic, underline.
- Middleware-like wrappers.

Common mistakes

- Do not use it when only one fixed extension is needed.
- Too many wrappers can make debugging harder.
- Do not confuse it with inheritance-only extension.

Java example

```
interface Coffee {
    int cost();
    String description();
}

class SimpleCoffee implements Coffee {
    public int cost() { return 50; }
    public String description() { return "Simple coffee"; }
}

abstract class CoffeeDecorator implements Coffee {
    protected final Coffee coffee;

    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }
}

class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) { super(coffee); }
    public int cost() { return coffee.cost() + 20; }
    public String description() { return coffee.description() + ", milk"; }
}

class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) { super(coffee); }
    public int cost() { return coffee.cost() + 10; }
    public String description() { return coffee.description() + ", sugar"; }
```

```
}

class Main {
    public static void main(String[] args) {
        Coffee coffee = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
        System.out.println(coffee.description() + " = " + coffee.cost());
    }
}
```

8. Adapter Pattern

Core idea: It makes an incompatible external or legacy interface work with the interface your system expects.

Question	Answer
Symptom in code	Third-party SDK methods and data formats leak into business logic.
Mental model	A translator between your clean interface and someone else's interface.
Interview sentence	I will use Adapter Pattern because it makes an incompatible external or legacy interface work with the interface your system expects.

When to use it

- Integrating third-party APIs.
- Vendor method names or formats do not match your design.
- You may switch vendors later.
- You want to isolate external dependencies.

Where it appears in LLD

- Razorpay, Stripe, PayU adapters.
- Twilio, MSG91 SMS adapters.
- S3, GCP, Azure storage adapters.
- Google, Apple, GitHub auth adapters.

Common mistakes

- Do not put business rules inside adapter.
- Adapter should translate, not own the full workflow.
- Avoid exposing vendor objects to the rest of the system.

Java example

```
interface PaymentProcessor {
    void pay(double amountInRupees);
}

class RazorpayGateway {
    public void makePaymentInPaise(int amountInPaise) {
        System.out.println("Razorpay paid paise: " + amountInPaise);
    }
}

class RazorpayAdapter implements PaymentProcessor {
    private final RazorpayGateway gateway;

    public RazorpayAdapter(RazorpayGateway gateway) {
        this.gateway = gateway;
    }

    public void pay(double amountInRupees) {
        int amountInPaise = (int) (amountInRupees * 100);
        gateway.makePaymentInPaise(amountInPaise);
    }
}
```

9. Chain of Responsibility Pattern

Core idea: It passes a request through a chain of handlers where each handler can process, reject, or forward it.

Question	Answer
Symptom in code	One method contains auth, validation, rate limit, logging, approval, and business processing together.
Mental model	A pipeline of independent handlers.
Interview sentence	I will use Chain of Responsibility Pattern because it passes a request through a chain of handlers where each handler can process, reject, or forward it.

When to use it

- Request needs multiple processing steps.
- Each step can stop the request.
- Steps should be reusable and reorderable.
- The workflow looks like middleware, approval chain, or validation pipeline.

Where it appears in LLD

- API middleware: auth, validation, rate limit.
- Logger levels: error, warning, info.
- ATM cash dispenser.
- Leave approval chain.
- Order validation pipeline.

Common mistakes

- Do not hide important order dependencies.
- Do not make handlers too large.
- Ensure the chain terminates correctly.

Java example

```
abstract class Handler {
    private Handler next;

    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    public void handle(String request) {
        if (next != null) {
            next.handle(request);
        }
    }
}

class AuthHandler extends Handler {
    public void handle(String request) {
        if ("NO_AUTH".equals(request)) {
            System.out.println("Rejected: auth failed");
            return;
        }
        System.out.println("Auth passed");
        super.handle(request);
    }
}

class ValidationHandler extends Handler {
```

```
    public void handle(String request) {
        System.out.println("Validation passed");
        super.handle(request);
    }
}

class BusinessHandler extends Handler {
    public void handle(String request) {
        System.out.println("Business logic executed");
    }
}
```

10. Repository Pattern

Core idea: It separates data access logic from business logic through repository interfaces.

Question	Answer
Symptom in code	Services contain SQL queries, database maps, storage details, and business rules together.
Mental model	Services ask repositories for data; repositories handle persistence.
Interview sentence	I will use Repository Pattern because it separates data access logic from business logic through repository interfaces.

When to use it

- You save, fetch, update, or delete domain objects.
- You want business logic to be testable.
- You may switch database/storage later.
- You want clean service classes.

Where it appears in LLD

- UserRepository, BookingRepository, ShowRepository.
- OrderRepository in food delivery.
- ExpenseRepository in Splitwise.
- InspectionRepository, ReportRepository, ImageRepository in audit apps.

Common mistakes

- Repository should not contain heavy business rules.
- Do not let services know DB-specific query details.
- Use interfaces so tests can use in-memory repositories.

Java example

```
import java.util.*;

class User {
    private final String id;
    private final String name;

    public User(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getId() { return id; }
    public String getName() { return name; }
}

interface UserRepository {
    void save(User user);
    User findById(String id);
}

class InMemoryUserRepository implements UserRepository {
    private final Map<String, User> store = new HashMap<>();

    public void save(User user) {
        store.put(user.getId(), user);
    }

    public User findById(String id) {
```

```
        return store.get(id);
    }
}

class UserService {
    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public void createUser(String id, String name) {
        repository.save(new User(id, name));
    }
}
```

11. Command Pattern

Core idea: It wraps an operation/request as an object so it can be executed, queued, retried, logged, or undone.

Question	Answer
Symptom in code	Actions are called directly, but now you need history, undo, retry, scheduling, or queuing.
Mental model	Action as an object.
Interview sentence	I will use Command Pattern because it wraps an operation/request as an object so it can be executed, queued, retried, logged, or undone.

When to use it

- You need undo/redo.
- You need a job queue or task scheduler.
- You need action history or audit logs.
- You want to decouple invoker from receiver.

Where it appears in LLD

- Text editor commands: type, delete, paste.
- Trading commands: buy, sell, cancel.
- Job scheduler commands.
- Remote control button actions.
- Order commands: place, cancel, refund.

Common mistakes

- Do not create command classes for every trivial method.
- Keep command focused on one action.
- For undo, store enough previous state.

Java example

```
interface Command {
    void execute();
    void undo();
}

class Light {
    public void on() { System.out.println("Light on"); }
    public void off() { System.out.println("Light off"); }
}

class LightOnCommand implements Command {
    private final Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() { light.on(); }
    public void undo() { light.off(); }
}

class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }
```

```
    public void press() { command.execute(); }
    public void undo() { command.undo(); }
}
```

12. Facade Pattern

Core idea: It provides a simple entry point over a complex subsystem or multi-step workflow.

Question	Answer
Symptom in code	Client/controller must call many services in the right order to complete one use case.
Mental model	One clean method outside; many coordinated services inside.
Interview sentence	I will use Facade Pattern because it provides a simple entry point over a complex subsystem or multi-step workflow.

When to use it

- A workflow involves multiple services.
- Client should not know subsystem complexity.
- You want one stable API for a complex use case.
- You want orchestration in one place.

Where it appears in LLD

- BookingFacade in BookMyShow.
- OrderFacade in food delivery.
- PaymentFacade around gateway, fraud, ledger.
- ReportGenerationFacade in audit/report apps.
- TripBookingFacade for flight, hotel, cab.

Common mistakes

- Facade should coordinate; avoid dumping all business logic into it.
- Do not hide errors too much.
- Keep underlying services independently testable.

Java example

```
class SeatService {
    void lockSeat() { System.out.println("Seat locked"); }
}

class PaymentService {
    void pay() { System.out.println("Payment done"); }
}

class TicketService {
    void generateTicket() { System.out.println("Ticket generated"); }
}

class NotificationService {
    void notifyUser() { System.out.println("User notified"); }
}

class BookingFacade {
    private final SeatService seatService = new SeatService();
    private final PaymentService paymentService = new PaymentService();
    private final TicketService ticketService = new TicketService();
    private final NotificationService notificationService = new NotificationService();

    public void bookTicket() {
        seatService.lockSeat();
        paymentService.pay();
        ticketService.generateTicket();
        notificationService.notifyUser();
    }
```

```
}
```

13. Composite Pattern

Core idea: It models tree structures so individual objects and groups can be treated through the same interface.

Question	Answer
Symptom in code	You separately handle files and folders, employees and managers, menu items and menus, products and bundles.
Mental model	Part-whole hierarchy. Leaf and container share the same interface.
Interview sentence	I will use Composite Pattern because it models tree structures so individual objects and groups can be treated through the same interface.

When to use it

- You have a tree or hierarchy.
- A container can contain both items and other containers.
- The same operation should work on a single item and a group.
- Recursive structure appears in the problem.

Where it appears in LLD

- File system: file and folder.
- Organization chart: employee and manager.
- Menu and submenu.
- UI component tree.
- Product bundles.

Common mistakes

- Do not use Composite for flat lists.
- Keep operations meaningful for both leaf and composite.
- Be careful with recursive cycles.

Java example

```
import java.util.*;

interface FileSystemComponent {
    void show(String indent);
}

class FileItem implements FileSystemComponent {
    private final String name;

    public FileItem(String name) {
        this.name = name;
    }

    public void show(String indent) {
        System.out.println(indent + "File: " + name);
    }
}

class Folder implements FileSystemComponent {
    private final String name;
    private final List<FileSystemComponent> children = new ArrayList<>();

    public Folder(String name) {
        this.name = name;
    }
```

```
    public void add(FileSystemComponent component) {
        children.add(component);
    }

    public void show(String indent) {
        System.out.println(indent + "Folder: " + name);
        for (FileSystemComponent child : children) {
            child.show(indent + "  ");
        }
    }
}
```

How to apply patterns in a real LLD interview

Example: BookMyShow

- Repository: UserRepository, MovieRepository, ShowRepository, BookingRepository.
- Strategy: PaymentStrategy and SeatSelectionStrategy.
- Factory: PaymentFactory creates the selected payment strategy.
- State: Booking moves through CREATED, PAYMENT_PENDING, CONFIRMED, CANCELLED, EXPIRED.
- Observer: Booking confirmation triggers email, SMS, analytics, and ticket delivery.
- Facade: BookingFacade coordinates seat lock, payment, booking confirmation, and notification.

Example: AMIRA structural audit app

- Repository: InspectionRepository, ReportRepository, ImageRepository, DefectRepository.
- Strategy: Severity calculation strategies for crack, corrosion, dampness, deflection, and settlement.
- Factory: TestFactory creates test modules such as rebound hammer, UPV, half-cell potential, carbonation, cover meter.
- Builder: AuditReportBuilder creates a final report with client details, site details, readings, images, summary, signature, and appendices.
- State: Report lifecycle can move through DRAFT, IN_PROGRESS, REVIEW_PENDING, APPROVED, SIGNED, SUBMITTED.
- Adapter: S3StorageAdapter, ESignAdapter, PaymentGatewayAdapter, AIModelAdapter.
- Facade: ReportGenerationFacade coordinates data fetching, AI analysis, PDF generation, e-signing, storage, and notification.

Final interview rule

Do not force patterns. First explain the design problem, then introduce the pattern naturally. For example: "Payment behavior can vary, so I will define a PaymentStrategy interface and implement UPI, card, and wallet separately."